

Lineare Algebra - Prüfung 1 - Lösung

- Studiengänge: MEC-VZ, EIT-DU, UMW der FH Vorarlberg
- Datum: 2025-04-28
- Uhrzeit: 08:50 - 10:30 Uhr

WICHTIG:

1. Die Bearbeitungen der Aufgaben müssen nachvollziehbar dargestellt sein.
2. Bei mit PYTHON gekennzeichneten Aufgaben können Sie Nicht-Code-Antworten als Kommentare in den Code-Zellen oder in eigenen Markdown-Zellen angeben.
3. Die erlaubten Unterlagen umfassen einen doppelseitig beschriebene A4-Zettel als Formelsammlung und die Datei `python-tutorial.pdf`.

BITTE HIER DEN NAMEN EINTRAGEN:

Aufgabe 1: Lineare Gleichungssysteme (2 + 2 = 4 Punkte)

Gegeben sei das folgende lineare Gleichungssystem:

$$x_1 + 2x_2 + x_3 + 2x_4 = 3$$

$$2x_1 + 4x_2 + x_3 + 3x_4 = 4$$

$$3x_1 + 6x_2 + x_3 + 4x_4 = 5$$

1. PAPIER: Bestimmen Sie mit Hilfe des Gaußschen Eliminationsverfahrens die Lösungsmenge des Gleichungssystems.
2. PYTHON: Bestimmen Sie den Rang der Koeffizientenmatrix und den Rang der erweiterten Koeffizientenmatrix, und schließen Sie damit auf die Lösungsstruktur des linearen Gleichungssystems. Schreiben Sie Ihre Antwort in einem Python-Kommentar.

Lösung

1. PAPIER: Das GEV liefert, dass das Gleichungssystem unendlich viele Lösungen hat. Zum Beispiel: x_4 ist frei wählbar, $x_3 = 2 - x_4$, x_2 ist auch frei wählbar, $x_1 = 1 - 2x_2 - x_4$. Siehe auch die Codeausgabe.
2. PYTHON: Siehe Code, von dem nur die Rangabfragen und die Antworten relevant sind.

```
import numpy as np
import matplotlib.pyplot as plt

def my_print_Ab(Ab):
    (m, n_plus_1) = np.shape(Ab)
    n = n_plus_1 - 1
    for i in range(m):
        for j in range(n):
```

```

        print(f"{Ab[i,j]:2.0f}", end = " ")
    print(f"|{Ab[i,n]:2.0f}")

Ab = np.array([[1, 2, 1, 2, 3],
               [2, 4, 1, 3, 4],
               [3, 6, 1, 4, 5]])
Ab_orig = Ab.copy()
print('erweiterte Koeffizientenmatrix:')
my_print_Ab(Ab)

# Z2 - 2*Z1:
Ab[1] = Ab[1] - 2*Ab[0]
print('erweiterte Koeffizientenmatrix nach 1. Zeilenaddition:')
my_print_Ab(Ab)

# Z3 - 3*Z1:
Ab[2] = Ab[2] - 3*Ab[0]
print('erweiterte Koeffizientenmatrix nach 2. Zeilenaddition:')
my_print_Ab(Ab)

# Z3 - 2*Z2:
Ab[2] = Ab[2] - 2*Ab[1]
print('erweiterte Koeffizientenmatrix nach 3. Zeilenaddition:')
my_print_Ab(Ab)

print(f'Anzahl der Variablen = {np.shape(Ab_orig)[1] - 1}')
print(f'Rang der Koeffizientenmatrix =
{np.linalg.matrix_rank(Ab_orig[:, :-1])}')
print(f'Rang der erweiterten Koeffizientenmatrix =
{np.linalg.matrix_rank(Ab_orig)}')
print('Anzahl der freien Parameter = 2')

# Antwort: Unendlich viele Lösungen, da der Rang der Koeffizientenmatrix
kleiner ist als der Rang der erweiterten Koeffizientenmatrix.

```

```

erweiterte Koeffizientenmatrix:
1 2 1 2 | 3
2 4 1 3 | 4
3 6 1 4 | 5
erweiterte Koeffizientenmatrix nach 1. Zeilenaddition:
1 2 1 2 | 3
0 0 -1 -1 | -2
3 6 1 4 | 5
erweiterte Koeffizientenmatrix nach 2. Zeilenaddition:
1 2 1 2 | 3
0 0 -1 -1 | -2
0 0 -2 -2 | -4

```

erweiterte Koeffizientenmatrix nach 3. Zeilenaddition:

```
1 2 1 2 | 3
0 0 -1 -1 | -2
0 0 0 0 | 0
```

Anzahl der Variablen = 4

Rang der Koeffizientenmatrix = 2

Rang der erweiterten Koeffizientenmatrix = 2

Anzahl der freien Parameter = 2

Aufgabe 2: Lineare Abbildungen (2 + 3 = 5 Punkte)

Betrachten Sie die Abbildungsmatrix

$$A = \begin{pmatrix} \cos(\alpha) & \sin(\alpha) \\ -\sin(\alpha) & \cos(\alpha) \end{pmatrix}.$$

1. PYTHON: Berechnen Sie das Bild der Standardbasisvektoren $e_1 = (1, 0)^T$ und $e_2 = (0, 1)^T$ unter der Abbildung $f(x) = Ax$ für $\alpha = \frac{\pi}{4}$. Welche Operation/Transformation beschreibt die Abbildung? Schreiben Sie Ihre Antwort in einem Python-Kommentar.
2. PAPIER: Zeigen Sie von Hand, dass $A^T A = I$ gilt, und berechnen Sie $\det(A)$ sowie A^{-1} .

Hinweis: π können Sie in Python mit `np.pi` angeben.

Lösung

```
alpha = np.pi/4
A = np.array([[ np.cos(alpha), np.sin(alpha)],
               [-np.sin(alpha), np.cos(alpha)]])

e1 = np.array([[1], [0]])
e2 = np.array([[0], [1]])

print(f"Ae1 = \n{A@e1}")
print(f"Ae2 = \n{A@e2}")

# R ist eine Rotation um alpha im Uhrzeigersinn.
```

```
Ae1 =
[[ 0.70710678]
 [-0.70710678]]
Ae2 =
[[0.70710678]
 [0.70710678]]
```

$$A^T A = \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{pmatrix} \begin{pmatrix} \cos(\alpha) & \sin(\alpha) \\ -\sin(\alpha) & \cos(\alpha) \end{pmatrix} = \begin{pmatrix} \cos^2(\alpha) + \sin^2(\alpha) & 0 \\ 0 & \cos^2(\alpha) + \sin^2(\alpha) \end{pmatrix} = I.$$

$$\det(A) = \cos^2(\alpha) + \sin^2(\alpha) = 1$$

$A^{-1} = \begin{pmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{pmatrix}$, was übrigens gleich A^T und $A(-\alpha)$ ist.

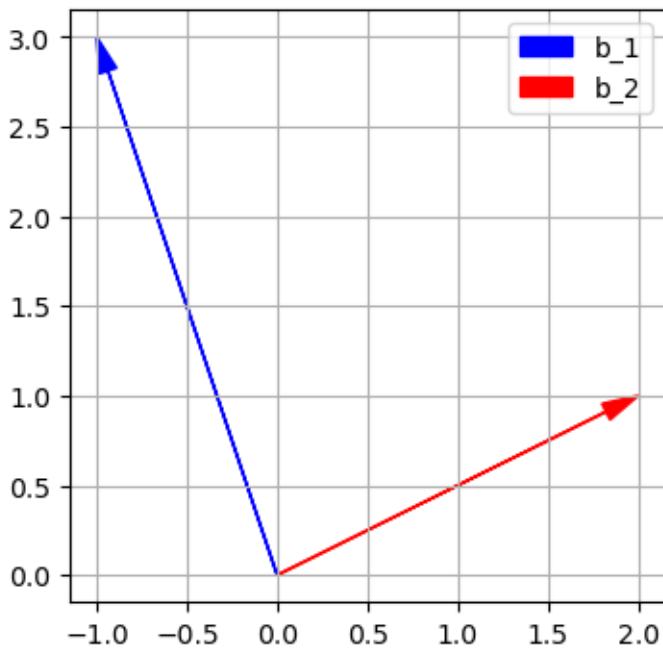
Aufgabe 3: Darstellung in einer Basis (1 + 1 + 2 = 4 Punkte)

Gegeben ist der Vektor $v = \begin{pmatrix} 7 \\ 0 \end{pmatrix}$ und die Basis $B = \begin{pmatrix} -1 & 2 \\ 3 & 1 \end{pmatrix}$.

1. PAPIER: Zeichnen Sie die Basisvektoren, d. h. die Spaltenvektoren von B , in ein Koordinatensystem.
2. PAPIER: Zeigen Sie durch eine passende Rechnung, dass B eine Basis ist.
3. PYTHON: Stellen Sie v in der Basis B dar, indem Sie ein entsprechendes LGS lösen, um die Koordinaten von v bzgl. der Basis B zu bestimmen.

Lösung

```
plt.figure(figsize=(4, 4))
plt.arrow(0, 0, -1, 3, head_width=0.1, head_length=0.2, fc='blue', ec='blue',
label='b_1', length_includes_head=True)
plt.arrow(0, 0, 2, 1, head_width=0.1, head_length=0.2, fc='red', ec='red',
label='b_2', length_includes_head=True)
plt.legend()
plt.grid(True)
```



B ist eine Basis, denn die Basisvektoren sind linear unabhängig, weil die Determinante -7 und somit nicht Null ist oder weil der Rang gleich zwei ist, und spannen den $\mathbb{R}^2$ auf. Das LGS $Bc = v$ hat die Lösung $c = \begin{pmatrix} -1 \\ 3 \end{pmatrix}$, also $v = -b_1 + 3 \cdot b_2$.

```

B = np.array([[ -1, 2],
              [ 3, 1]])
print(f"det(B) = {np.linalg.det(B):.1f}.")
print(f"rank(B) = {np.linalg.matrix_rank(B):.0f}.")
print("Die Basisvektoren sind daher linear unabhängig und B ist eine Basis.")
v = np.array([7, 0])
c = np.linalg.solve(B, v)
print(f"v = {c[0]}*b1 + {c[1]}*b2")

```

```

det(B) = -7.0.
rank(B) = 2.
Die Basisvektoren sind daher linear unabhängig und B ist eine Basis.
v = -1.0*b1 + 3.0*b2

```

Aufgabe 4: Eigenwerte und Eigenvektoren (3 + 2 = 5 Punkte)

1. PAPIER: Bestimmen Sie die Eigenwerte und Eigenvektoren der Matrix

$$\begin{pmatrix} -2 & -5 \\ 1 & 4 \end{pmatrix}.$$

2. PYTHON: Überprüfen Sie das Ergebnis in Python

1. mit der Funktion `np.linalg.eig()` und durch sinnvolles Skalierung der Eigenvektoren.
2. indem Sie überprüfen, ob die Eigenvektoren und Eigenwerte der PAPIER-Lösung die Eigenwertgleichung $Ax = \lambda x$ erfüllen.

Lösung

$\det(A - \lambda I) = \det\begin{pmatrix} -2-\lambda & -5 \\ 1 & 4-\lambda \end{pmatrix} = (-2-\lambda)(4-\lambda) + 5 = \lambda^2 - 2\lambda + 3$ hat die Nullstellen -1 und 3 . Diese zwei Lösungen λ_i von $\det(A - \lambda I) = 0$ sind die Eigenwerte von A : $\lambda_1 = -1$, $\lambda_2 = 3$. Die zwei dazugehörigen Eigenvektoren v_i sind die Lösungen der linearen Gleichungssysteme $(A - \lambda I)v_i = 0$: $v_1 = (-5, 1)^T$, $v_2 = (-1, 1)^T$. Genauer: $\begin{pmatrix} -2-(-1) & -5 \\ 1 & 4-(-1) \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = 0$ liefert $v_1 = (-5, 1)^T$ und $\begin{pmatrix} -2-3 & -5 \\ 1 & 4-3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = 0$ liefert $v_2 = (-1, 1)^T$.

```

A = np.array([[ -2, -5],
              [ 1, 4]])
L, V = np.linalg.eig(A)

print(f"eigenvalues = {L}")
print(f"rescaled 1st eigenvector: \n{V[:,0]}/V[1, 0]}")
print(f"rescaled 2nd eigenvector: \n{V[:,1]}/V[1, 1]}")

print(f"Eigenwertgleichung 1: A v_1 - lbd_1 v_1 = \n{A@V[:,0]} - L[0]*V[:,0]}")
print(f"Eigenwertgleichung 2: A v_2 - lbd_2 v_2 = \n{A@V[:,1]} - L[1]*V[:,1]}")

```

```

eigenvalues = [-1.  3.]
rescaled 1st eigenvector:
[[-5.]
 [ 1.]]
rescaled 2nd eigenvector:
[[-1.]
 [ 1.]]
Eigenwertgleichung 1: A v_1 - lbd_1 v_1 =
[[-1.11022302e-16]
 [ 1.11022302e-16]]
Eigenwertgleichung 2: A v_2 - lbd_2 v_2 =
[[0.]
 [0.]]

```

Aufgabe 5: Diskrete Dynamische Systeme (1 + 2 + 2 + 2 = 7 Punkte)

Ein diskretes dynamisches System ist durch die Übergangsmatrix

$$M = \begin{pmatrix} 0.9 & 0.2 & 0.0 \\ 0.1 & 0.5 & 0.2 \\ 0.0 & 0.3 & 0.8 \end{pmatrix}$$

und den Anfangszustand

$$x_0 = \begin{pmatrix} 100 \\ -200 \\ 300 \end{pmatrix}$$

gegeben.

1. PAPIER: Was ist ein diskretes dynamisches System? Was sind dabei die Rollen der Übergangsmatrix M und des Anfangszustands x_0 ?
2. PYTHON: Bestimmen Sie die Eigenwerte und -vektoren der Übergangsmatrix M , und bestimmen Sie die Koordinaten des Anfangszustands x_0 bzgl. der Eigenbasis.
3. PAPIER: Analysieren Sie das Langzeitverhalten des dynamischen Systems mit Hilfe der bisherigen Berechnungen. Wohin tendiert das System? Begründen Sie Ihre Antwort.
4. PYTHON: Überprüfen Sie Ihre Ergebnisse in Python, indem Sie die Entwicklung des Systems für 40 Schritte simulieren und plotten.

Lösung

Ein diskretes dynamisches System ist gegeben durch einen Anfangszustandsvektor x_0 und eine Übergangsmatrix M , die den Zustand des Systems in jedem Schritt transformiert. Der Anfangszustand x_0 beschreibt den Startpunkt des Systems, während die Matrix M die Regeln für die Transformation des Zustands in jedem Zeitschritt angibt.

```

# initial state:
x0 = np.array([[ 100],
               [-200],
               [ 300]])

# transition matrix:
M = np.array([[0.9, 0.2, 0.0],
              [0.1, 0.5, 0.2],
              [0.0, 0.3, 0.8]])

# eigenvalues and eigenvectors:
L, V = np.linalg.eig(M)
print(f"Eigenwerte = {L}")
print(f"Matrix mit Eigenvektoren in den Spalten = \n{V}")

y0 = np.linalg.solve(V, x0)
print(f"Koordinaten of x0 in der Eigenbasis = \n{y0}")

x_inf = y0[2,0]*V[:,[2]]
print(f"Langzeitzustand x_inf = \n{x_inf}")

```

```

Eigenwerte = [0.33542487 0.86457513 1.          ]
Matrix mit Eigenvektoren in den Spalten =
[[-0.28523152 -0.76505532  0.74278135]
 [ 0.8051731  0.13550992  0.37139068]
 [-0.51994159 0.6295454  0.55708601]]
Koordinaten of x0 in der Eigenbasis =
[[-321.30903495]
 [ 105.26866446]
 [ 119.67032905]]
Langzeitzustand x_inf =
[[88.88888889]
 [44.44444444]
 [66.66666667]]

```

Das System tendiert zu einem Vielfachenes Eigenvektors v_3 mit dem Eigenwert $\lambda_3 = 1$, da alle anderen Eigenwerte $|\lambda_i| < 1$. Genauer:

$$\begin{aligned}
 x_0 &= c_1 v_1 + c_2 v_2 + c_3 v_3 \\
 x_n &= M^n x_0 = M^n (c_1 v_1 + c_2 v_2 + c_3 v_3) \\
 &= c_1 M^n v_1 + c_2 M^n v_2 + c_3 M^n v_3 \\
 &= c_1 \lambda_1^n v_1 + c_2 \lambda_2^n v_2 + c_3 \lambda_3^n v_3 \\
 &\rightarrow 0 \cdot v_1 + 0 \cdot v_2 + c_3 v_3 \quad \text{für } n \rightarrow \infty
 \end{aligned}$$

```

# dynamics:
n = 40 # number of years
times = np.arange(n) # vector of years from now
X = np.zeros((3, n)) # matrix with states as columns

X[:,0] = x0 # put initial state into first column
for k in range(1, n): # iterate over years
    X[:,k] = M@X[:,k - 1]

plt.figure(figsize=(5, 4))
plt.plot(times, X[0,:], '-.', label='Komponente 1')
plt.plot(times, X[1,:], '-.', label='Komponente 2')
plt.plot(times, X[2,:], '-.', label='Komponente 3')
plt.legend(loc='best')
plt.xlabel('Zeit')
plt.ylabel('Zustand')
plt.grid(True)

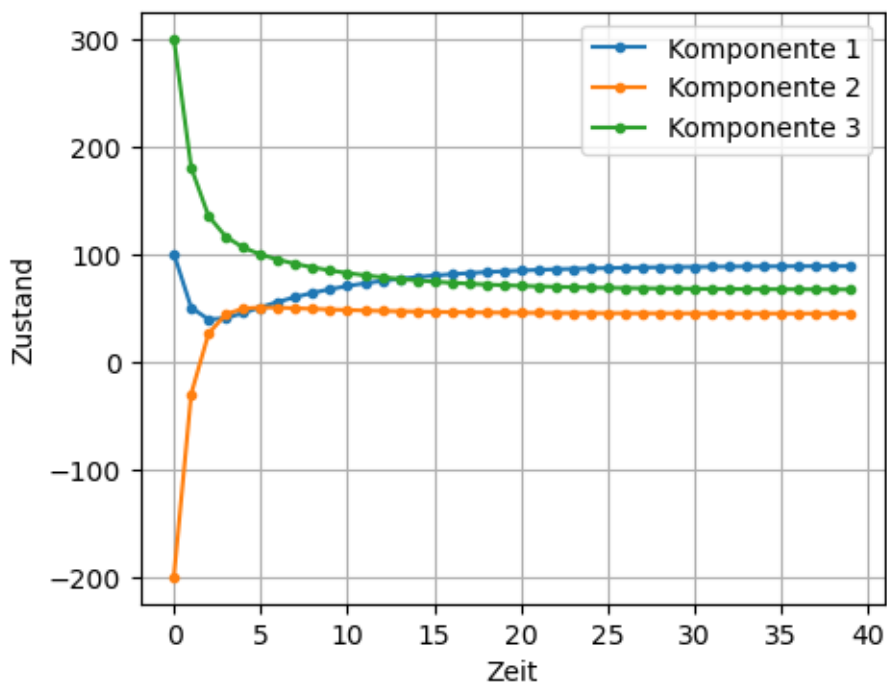
print(f"last state = \n{X[:, -1]}")

```

```

last state =
[[88.61264112]
 [44.49337465]
 [66.89398423]]

```



Aufgabe 6: Lineare Regression (1 + 1 + 1 + 2 = 5 Punkte)

Im Code unten sind die Matrix A und die Vektoren t und b gegeben.

1. PYTHON: Warum ist das lineare Gleichungssystem $Ax = b$ nicht lösbar?
2. PYTHON: Plotten Sie t (auf der x -Achse) gegen b (auf der y -Achse) inkl. Achsenbeschriftung.
3. PYTHON: Lösen Sie das OLS Problem $\min \|Ax - b\|$, und ergänzen Sie den Plot um den optimalen Fit und eine Legende.
4. PAPIER: Beantworten Sie folgende allgemeine Fragen:
 1. Warum ist ein OLS Problem $\min \|Ax - b\|$ immer lösbar? Argumentieren Sie geometrisch.
 2. Wann ist die Lösung eines OLS Problems $\min \|Ax - b\|$ auch eine Lösung des linearen Gleichungssystems $Ax = b$, und wann ist das nicht der Fall?

```
import numpy as np
np.random.seed(0)
n = 50
t = np.linspace(0, 5*np.pi, n)
b = 1.5 + np.sin(t) - 0.75*np.sin(2*t) + 0.1*np.random.randn(n)
b = b.reshape(-1, 1)
A = np.hstack((np.ones((n, 1)), np.vstack((np.sin(t), np.sin(2*t))).T))
```

Lösung

```
np.random.seed(0)
n = 50
t = np.linspace(0, 5*np.pi, n)
b = 1.5 + np.sin(t) - 0.75*np.sin(2*t) + 0.1*np.random.randn(n)
b = b.reshape(-1, 1)
A = np.hstack((np.ones((n, 1)), np.vstack((np.sin(t), np.sin(2*t))).T))
```

```
Ab = np.hstack((A, b))
print('Rang der Koeffizientenmatrix =', np.linalg.matrix_rank(A))
print('Rang der erweiterten Koeffizientenmatrix =', np.linalg.matrix_rank(Ab))
```

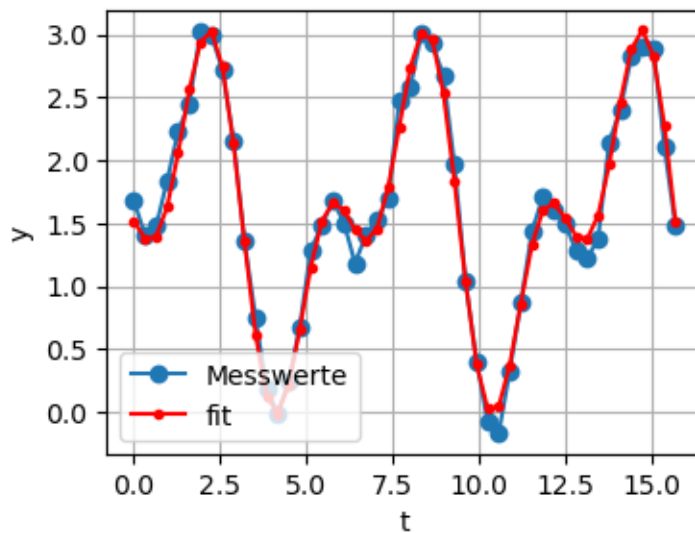
Antwort: Keine Lösung, da der Rang der Koeffizientenmatrix kleiner ist als der Rang der erweiterten Koeffizientenmatrix.

Rang der Koeffizientenmatrix = 3
Rang der erweiterten Koeffizientenmatrix = 4

```
x_opt = np.linalg.lstsq(A, b)[0]
b_fit = A @ x_opt

plt.figure(figsize=(4, 3))
plt.plot(t, b, 'o-', label='Messwerte')
```

```
plt.plot(t, b_fit, '.r-', label='fit')
plt.xlabel('t')
plt.ylabel('y')
plt.grid(True)
plt.legend(loc='best')
plt.show()
```



1. Ein OLS Problem $\min \|Ax - b\|$ ist immer lösbar, weil es geometrisch der orthogonalen Projektion des Vektors b auf den von den Spalten von A aufgespannten Raums entspricht, und eine solche Projektion liefert immer ein eindeutiges Ergebnis.
2. Die Lösung eines OLS Problems $\min \|Ax - b\|$ ist genau dann auch eine Lösung des linearen Gleichungssystems $Ax = b$, wenn das LGS $Ax = b$ überhaupt eine Lösung hat, d. h. wenn der Rang der Koeffizientenmatrix gleich dem Rang der erweiterten Koeffizientenmatrix ist.